ASSIGNMENT 13.1

V.Abhilash
HT : 2303A51922
BATCH 12

 # Task: Use AI to refactor a given Python script that contains mul code blocks.

 # Original code with repeated blocks

```python
# Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
# Original code with repeated blocks
def calculate_area(radius):
    area = 3.14 * radius * radius
    return area
def calculate_circumference(radius):
    circumference = 2 * 3.14 * radius
    return circumference
def calculate_diameter(radius):
    diameter = 2 * radius
    return diameter
# Refactored code using a single function to handle all calculations
def calculate_circle_properties(radius):
    area = 3.14 * radius * radius
    circumference = 2 * 3.14 * radius
    diameter = 2 * radius
    return area, circumference, diameter
# Example usage
radius = 5
area, circumference, diameter = calculate_circle_properties(radius)
print(f"Area: {area}")
print(f"Circumference: {circumference}")
print(f"Diameter: {diameter}")
```

OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    powershell

PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
Area: 78.5
Circumference: 31.400000000000002
Diameter: 10
PS C:\Users\ADMIN\Desktop\AI_AC LAB>
```

#Task 2: Use AI to analyze a Python script with nested loops and complex condi

 # Original code with nested loops and complex condi

```python
#Task: Use AI to analyze a Python script with nested loops and complex conditionals.
# Original code with nested loops and complex conditionals
def process_data(data):
    result = []
    for item in data:
        if isinstance(item, list):
            for sub_item in item:
                if sub_item > 10:
                    result.append(sub_item * 2)
                else:
                    result.append(sub_item)
        elif isinstance(item, dict):
            for key, value in item.items():
                if value > 5:
                    result.append(value * 3)
                else:
                    result.append(value)
        else:
            if item > 0:
                result.append(item * 4)
            else:
                result.append(item)
    return result
# Refactored code using a single function to handle all cases
def process_data(data):
    result = []
    for item in data:
        if isinstance(item, list):
            result.extend([sub_item * 2 if sub_item > 10 else sub_item for sub_item in item])
        elif isinstance(item, dict):
            result.extend([value * 3 if value > 5 else value for value in item.values()])
```

OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                    powershell

PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
Area: 78.5
Circumference: 31.400000000000002
Diameter: 10
PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
[4, -2, 5, 30, 3, 21]
PS C:\Users\ADMIN\Desktop\AI_AC LAB>
```

#Task 3: Use AI to refactor a legacy script where mul
inside the main code block

 # Original code with calcula

```python
#Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block
# Original code with calculations embedded in the main code block
def main():
    radius = 5
    area = 3.14 * radius * radius
    circumference = 2 * 3.14 * radius
    diameter = 2 * radius
    print(f"Area: {area}")
    print(f"Circumference: {circumference}")
    print(f"Diameter: {diameter}")
# Refactored code with calculations moved to a separate function
def calculate_circle_properties(radius):
    area = 3.14 * radius * radius
    circumference = 2 * 3.14 * radius
    diameter = 2 * radius
    return area, circumference, diameter
def main():
    radius = 5
    area, circumference, diameter = calculate_circle_properties(radius)
    print(f"Area: {area}")
    print(f"Circumference: {circumference}")
    print(f"Diameter: {diameter}")
if __name__ == "__main__":
    main()
```

OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    powershell + ⌄

PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
Area: 78.5
Circumference: 31.400000000000002
Diameter: 10
PS C:\Users\ADMIN\Desktop\AI_AC LAB>
```

Task4 : Use AI to iden numbers" in the code with named constants.

```python
# Task: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants.
# Original code with hardcoded magic numbers
def calculate_area(radius):
    area = 3.14 * radius * radius
    return area
def calculate_circumference(radius):
    circumference = 2 * 3.14 * radius
    return circumference
def calculate_diameter(radius):
    diameter = 2 * radius
    return diameter
#Refactored code with named constants
PI = 3.14
def calculate_area(radius):
    area = PI * radius * radius
    return area
def calculate_circumference(radius):
    circumference = 2 * PI * radius
    return circumference
def calculate_diameter(radius):
    diameter = 2 * radius
    return diameter
# Example usage
radius = 5
area = calculate_area(radius)
circumference = calculate_circumference(radius)
diameter = calculate_diameter(radius)
print(f"Area: {area}")
print(f"Circumference: {circumference}")
print(f"Diameter: {diameter}")
```

OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   powershell

PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
Area: 78.5
Circumference: 31.400000000000002
Diameter: 10
PS C:\Users\ADMIN\Desktop\AI_AC LAB>
```

 # Task 5: Use AI to improve readability by renaming unclear variables and adding inline comments.

 # Original code with unclear variable names and no comments

```python

# Task 5: Use AI to improve readability by renaming unclear variables and adding inline comments.
# Original code with unclear variable names and no comments
def calc(r):
    a = 3.14 * r * r
    c = 2 * 3.14 * r
    d = 2 * r
    return a, c, d
# Refactored code with improved variable names and inline comments
PI = 3.14
def calculate_circle_properties(radius):
    # Calculate the area of the circle
    area = PI * radius * radius
    # Calculate the circumference of the circle
    circumference = 2 * PI * radius
    # Calculate the diameter of the circle
    diameter = 2 * radius
    return area, circumference, diameter
# Example usage
radius = 5
area, circumference, diameter = calculate_circle_properties(radius)
print(f"Area: {area}")
print(f"Circumference: {circumference}")
print(f"Diameter: {diameter}")

```

Task 6: Use AI to refactor a Python script that contains repeated if–else logic for grading students.

```python
#Task6: Use AI to refactor a Python script that contains repeated if-else logic for grading students.
# Original code with repeated if-else logic
def grade_student(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
# # Refactored code using a single function to handle grading logic
def grade_student(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
```

OUTPUT

```python
# Example usage
scores = [95, 85, 75, 65, 55]
grades = [grade_student(score) for score in scores]
print(grades)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                    powershell
.py"
PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
['A', 'B', 'C', 'D', 'F']
PS C:\Users\ADMIN\Desktop\AI_AC LAB>
```

• Task:7 Use AI to refactor procedural input–processing logic into fun

```python
# #Task: Use AI to refactor procedural input-processing logic into functions.
# Original code with procedural input-processing logic
# name = input("Enter your name: ")
# age = int(input("Enter your age: "))
# if age >= 18:
#     print(f"Hello {name}, you are an adult.")
# else:
#     print(f"Hello {name}, you are a minor.")
# Refactored code with input-processing logic moved to a function
def process_user_input():
    name = input("Enter your name: ")
    age = int(input("Enter your age: "))
    if age >= 18:
        print(f"Hello {name}, you are an adult.")
    else:
        print(f"Hello {name}, you are a minor.")
if __name__ == "__main__":
    process_user_input()
```

OUTPUT

```python
def process_user_input():
    name = input("Enter your name: ")
    age = int(input("Enter your age: "))
    if age >= 18:
        print(f"Hello {name}, you are an adult.")
    else:
        print(f"Hello {name}, you are a minor.")
if __name__ == "__main__":
    process_user_input()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                    powershell
PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
Enter your name: alexa
Enter your age: 19
Hello alexa, you are an adult.
PS C:\Users\ADMIN\Desktop\AI_AC LAB>
```

Task 8: Use AI to refactor inefficient list processing logic.

```python

# task: Use AI to refactor inefficient list processing logic.
# Original code with inefficient list processing logic
# def process_list(input_list):
#     output_list = []
#     for item in input_list:
#         if item % 2 == 0:
#             output_list.append(item * 2)
#         else:
#             output_list.append(item * 3)
#     return output_list
# Refactored code using list comprehension for efficient list processing
def process_list(input_list):
    return [item * 2 if item % 2 == 0 else item * 3 for item in input_list]
# Example usage
input_list = [1, 2, 3, 4, 5]
output_list = process_list(input_list)
print(output_list)
```

OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                    powershell

PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
Enter your name: alexa
Enter your age: 19
Hello alexa, you are an adult.
PS C:\Users\ADMIN\Desktop\AI_AC LAB> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/Desktop/AI_AC LAB/
.py"
[3, 4, 9, 8, 15]
PS C:\Users\ADMIN\Desktop\AI_AC LAB>
```